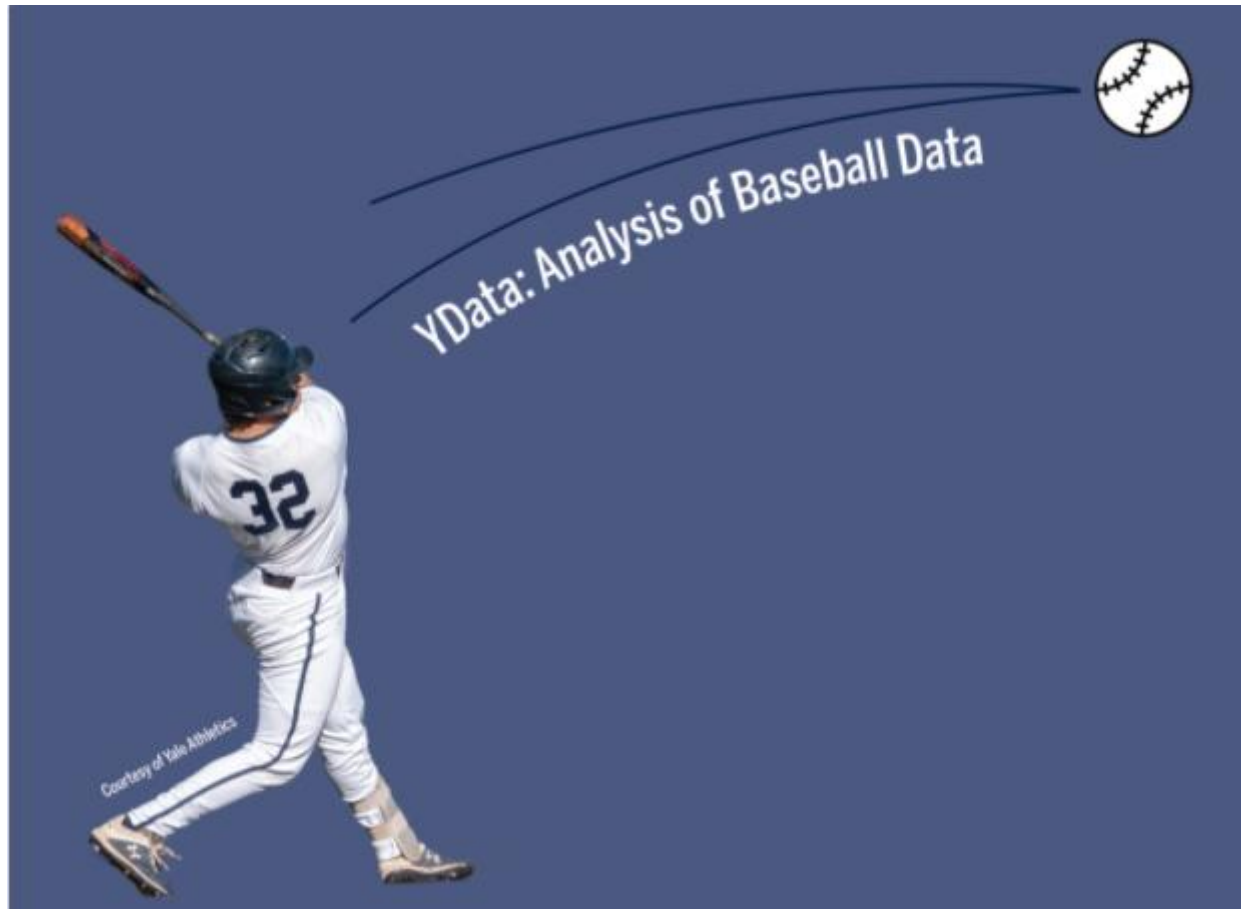


S&DS 1730



Class 12: SQL and Statcast data

Overview

Databases and Structured Query Language (SQL)

Statcast data

If there is time: work on projects

Final projects!

Use the Data Science methods you have learned in this class to answer a question related to baseball!

You will give a 2-minute (one slide) presentation on your project during class on April 20th

Final project written reports are due at 11pm on April 26th

- Report should be a Jupyter notebook approximately 7 pages long
- A final project Jupyter notebook template has been uploaded to the class GitHub site



Project presentations

You will give a 2-minute presentation about your project next class

- Explain what your project aims to show
- Show one or two preliminary results

You should submit a single PowerPoint slide on Canvas by **Sunday April 26th**

- The slide should show one or two interesting preliminary results

The baseball season has started!

American League East

Team	W	L	Pct	GB	Home	Away	L10
 Orioles	8	7	.533	-	5-4	3-3	6-4
 Yankees	8	7	.533	-	3-3	5-4	4-6
 Rays	8	7	.533	-	4-2	4-5	6-4
 Red Sox	6	9	.400	2.0	3-3	3-6	5-5
 Blue Jays	6	9	.400	2.0	6-6	0-3	2-8

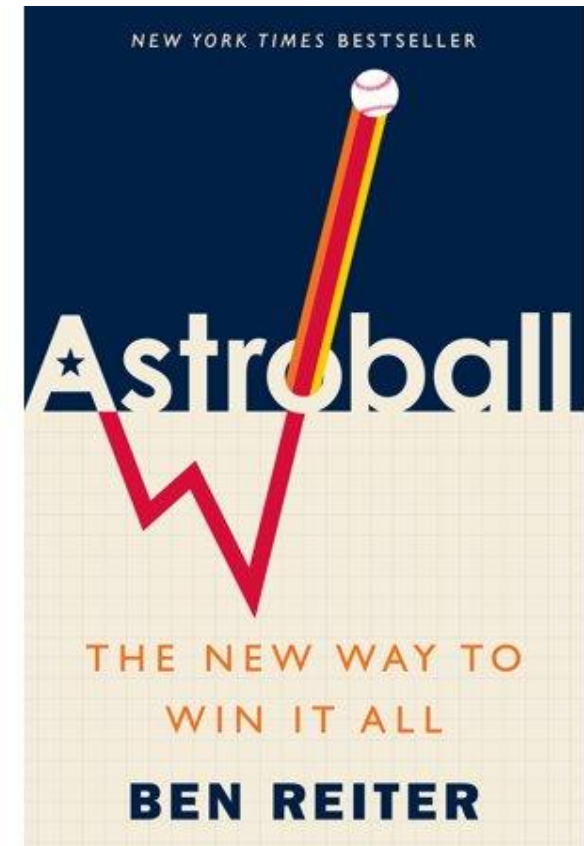
National League West

Team	W	L	Pct	GB	Home	Away	L10
 Dodgers	11	4	.733	-	6-3	5-1	7-3
 Padres	10	6	.625	1.5	6-4	4-2	8-2
 Diamondbacks	9	7	.563	2.5	5-2	4-5	6-4
 Rockies	6	10	.375	5.5	4-2	2-8	4-6
 Giants	6	10	.375	5.5	3-7	3-3	4-6

Discussion on chapter 9 of Astrobball

Let's discuss the chapter 9 for 3 minutes in small groups of 3-4 people and then have a larger conversation as a group

- E.g., Discuss your quote and reaction to chapter 9



Astrobball chapter 9

Discussion of game 7 of the world series:

"There was no doubt that the Astros' hitters were talented. On this night, it was almost as if they *knew* what was coming.

In fact they did."

- Astrobball page 209

[Correa proposing](#)

Ben Reiter will be visiting next class

- Please think of questions you would like to ask him!

Brief introduction to databases and SQL

Databases terminology

A **database** is an organized collection of data that users interact with using software called a database management system (DBMS)

Relational databases store data in a series of interrelated **tables**

Structured Query Language (SQL) is a programming language used for querying and updating the database

- Allows one to perform similar data manipulation operations that we have been doing using pandas

The vast majority of relational databases systems use SQL, however they often don't follow the ANSI standard exactly

- So sometimes it can require work update SQL queries from one system to another



Databases types

Some popular multi-user databases that use SQL include: Oracle, Microsoft SQL server, MySQL and PostgreSQL

- These systems use a client server architecture designed for many users querying and updating the system simultaneously

Other popular single user SQL systems are:

- **SQLite**: most widely used implementation of SQL partly because it is used to store data on Android phones
- **DuckDB**: single user SQL system designed for fast data analysis

We will focus on using DuckDB because it is fast easy to set up, but the SQL we learn to execute will be similar for other database systems

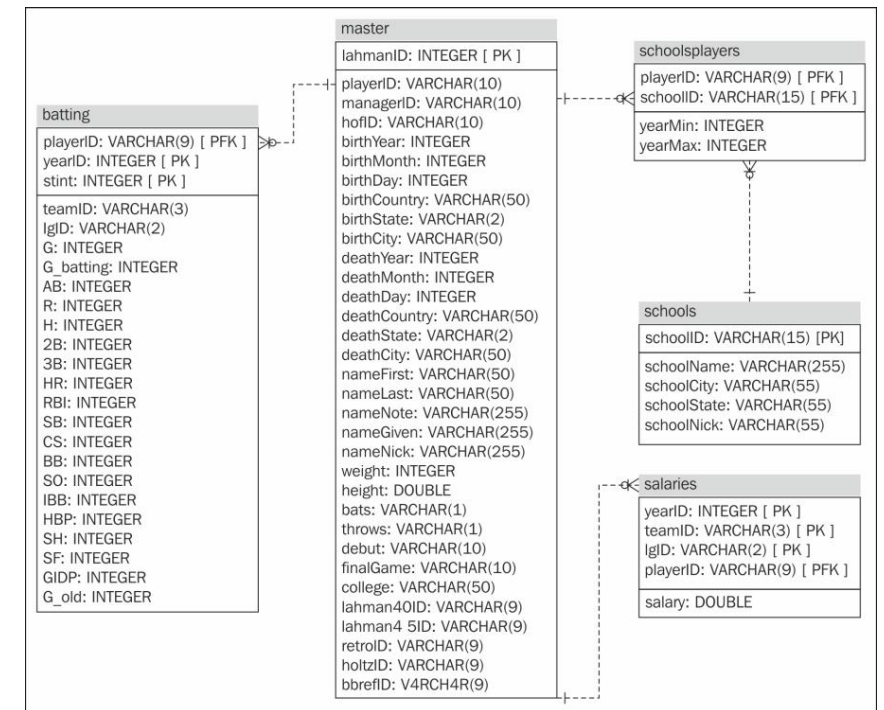


The Lahman Database

The Lahman database contains season level major league batting and pitching statistics back to 1871

Tables in the Lahman Database include:

- Batting
- Pitching
- People
- Salaries
- Awards
- etc.



We already have been using many of these tables which we have loaded from .csv files

Connecting and extracting data

As a first step we need to connect to our database using a database management system (DBMS)

We can do this in Python (and duckDB) using the `.connect()` function

```
import duckdb  
con = duckdb.connect("lahman_2025.duckdb")
```

Once we have connected, we can use the `.execute()` method to run SQL commands

We can use the following methods to extract data from the database:

- `.fetchall()` Returns data as a list of tuples
- `.fetchdf()` Returns data as a pandas DataFrame



Getting information about the database

To list all tables in a database we can use `SHOW TABLES` command

- `con.execute("SHOW TABLES").fetchdf()`

To get information about the columns in a particular table we can use the `DESCRIBE` command

- `con.execute("DESCRIBE Batting").fetchdf()`

Let's try it in Jupyter!

SQL Queries

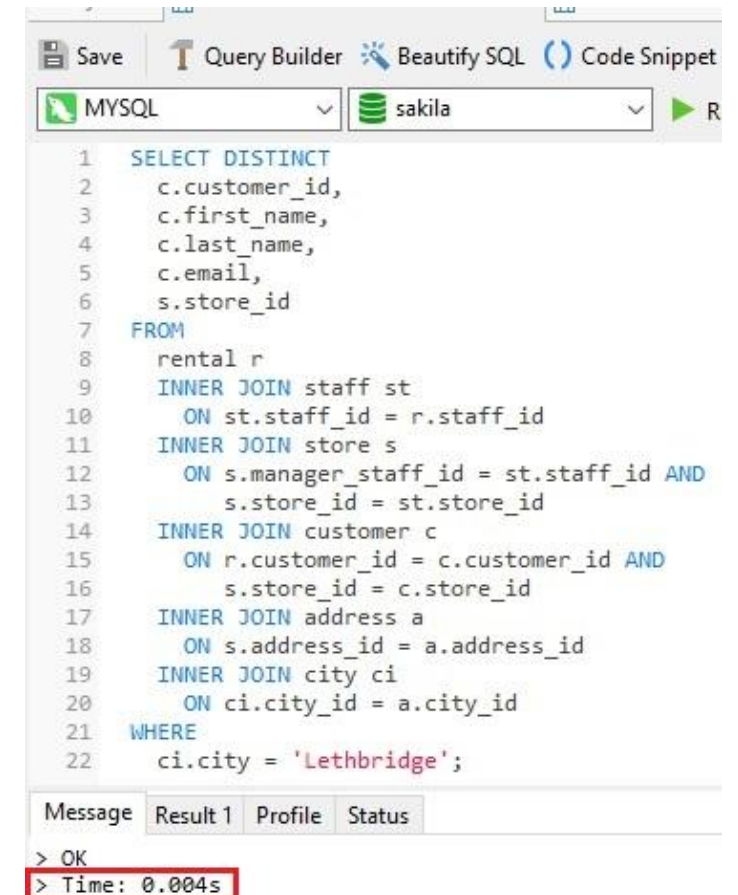
Queries in SQL start with a **SELECT** statement, and must be **written** in this order:

- SELECT
- FROM
- JOIN
- WHERE
- GROUP BY
- HAVING
- ORDER BY
- LIMIT

Queries are **evaluated** in the order:

- FROM
- JOIN
- WHERE
- GROUP BY
- **SELECT**
- ORDER BY

Usually, SQL statement must end with a semicolon ;



The screenshot shows a MySQL Query Builder window. The top toolbar includes buttons for 'Save', 'Query Builder', 'Beautify SQL', and 'Code Snippet'. Below the toolbar, there are dropdown menus for 'MYSQL' and 'sakila', followed by a green play button and a red 'R' button. The main area displays a SQL query with line numbers 1 through 22. The query is a complex join involving rental, staff, store, customer, address, and city tables, filtered by city = 'Lethbridge'. At the bottom, there are tabs for 'Message', 'Result 1', 'Profile', and 'Status'. The 'Message' tab is active, showing a green 'OK' message and a red box containing the execution time: '> Time: 0.004s'.

```
1 SELECT DISTINCT
2   c.customer_id,
3   c.first_name,
4   c.last_name,
5   c.email,
6   s.store_id
7 FROM
8   rental r
9   INNER JOIN staff st
10    ON st.staff_id = r.staff_id
11  INNER JOIN store s
12    ON s.manager_staff_id = st.staff_id AND
13       s.store_id = st.store_id
14  INNER JOIN customer c
15    ON r.customer_id = c.customer_id AND
16       s.store_id = c.store_id
17  INNER JOIN address a
18    ON s.address_id = a.address_id
19  INNER JOIN city ci
20    ON ci.city_id = a.city_id
21 WHERE
22   ci.city = 'Lethbridge';
```

Message Result 1 Profile Status

> OK
> Time: 0.004s

SELECT...FROM

The **SELECT** keyword lists the columns (or functions operating on columns) that you want to retrieve

The **FROM** keyword specifies the table we want to retrieve the data from

The SELECT and FROM keywords are the only required keyword for every SQL query

Example: **SELECT** playerId, HR **FROM** Batting

SELECT...FROM

SELECT statements can include functions of other columns

```
SELECT concat(col1, ' ', col2)
```

```
SELECT sum(HR)
```

We can also give columns aliases which make them easier to refer to using the AS statement

```
SELECT playerId, yearID, H-"2B"-"3B"-HR AS singles FROM Batting;
```

Let's try it in Jupyter!

WHERE

- The **WHERE** clause allows you to filter the data to a subset of rows that meet particular criteria
 - Similar to the `.query()` method in pandas (and Boolean indexing)

We can filter on multiple criteria using the **AND** and the **OR** keywords

We can use the **BETWEEN** keyword to get data in a range of values

We can use the **IN** keyword to get values in a list of values

Examples

```
SELECT playerId, yearID, HR, SB FROM Batting WHERE HR >= 60 AND SB >= 40
```

```
SELECT playerId FROM Batting WHERE teamID IN ('BOS', 'NYA')
```

Let's try it in Jupyter!



GROUP BY

The **GROUP BY** clause allows you to aggregate records according to some criteria

- Similar to pandas `groupby()`

We also need to use aggregate functions in the SELECT statement that are applied separately to each group

- Only columns in aggregate functions or grouped columns can be selected
- Aggregate functions include: **COUNT, SUM, MAX, and AVG**

We can also group by multiple columns

- `SELECT yearID, lgID, sum(HR) FROM Teams GROUP BY yearID, lgID;`

Let's try it in Jupyter

ORDER BY

The **ORDER BY** keyword allows you to arrange the results based on values in particular rows

- Similar to the `.sort_values()` method in pandas

We can arrange the results in either:

- Ascending order (smallest to largest): use **ASC** keyword
- Descending order (largest to smallest): use **DESC** keyword

Example

```
SELECT yearID, teamID, HR FROM Teams ORDER BY HR DESC
```

LIMIT and SAMPLE

The **LIMIT** keyword restricts the number of results returned

- Similar to the pandas `.head()` method

We can also:

- Get a percentage of rows: **LIMIT 10%**
- Get a random sample of rows by using the **USING SAMPLE** keywords:
 - **USING SAMPLE 100 ROWS**

Let's try it in Jupyter!

HAVING

The **HAVING** keyword is similar to the **WHERE** keyword but allows you to do a second round of filtering on the result set

- i.e., can first get aggregated results using **GROUP BY** and can then filter these using **HAVING**

Example

```
SELECT sum(H)/sum(AB) AS avg, count(*) AS num_seasons  
FROM Teams GROUP BY teamID  
HAVING num_seasons >= 20
```

Let's try it in Jupyter!



JOIN

The **JOIN** keyword allows you to combine two or more tables based on a key

- Similar to the `.join()` and `.merge()` methods in pandas

Types of joins are:

- **JOIN**: Does in "inner join" where there must be matches in both tables
- **LEFT JOIN**: Keeps all rows in the left table
- **RIGHT JOIN**: Keeps all rows in the right table
- **CROSS JOIN**: Cartesian product of the two tables

JOIN

Table aliases allow you to abbreviate the names of tables which can save a fair amount of typing

- These aliases can be referenced anywhere in the query

Example:

```
SELECT p.nameFirst, p.nameLast, b.yearID, b.HR  
FROM Batting b  
LEFT JOIN People p ON b.playerID = p.playerID  
WHERE b.HR >= 60
```

Let's try it in Jupyter!

UNION

The **UNION** keyword allows you to join two separate queries

- Similar to the pandas `pd.concat([df1, df2], ignore_index=True)`

Syntax looks like:

```
(SELECT a b c FROM table1...)
```

```
UNION
```

```
(SELCT a b c FROM table2...)
```

Let's try it in Jupyter!



Statcast data

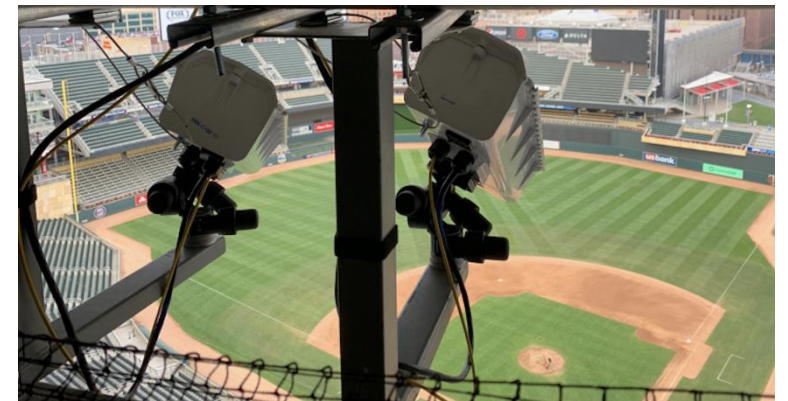
Statcast

Statcast is an automated tool developed to analyze player movements and athletic abilities

- Statcast was introduced to all thirty MLB stadiums in 2015

The system collects data through a series of cameras placed around the stadium

- These camera track the movement of the ten players on the field
- They also measure the speed of pitches and how the ball makes contact with the bat



Records 100 frames per second

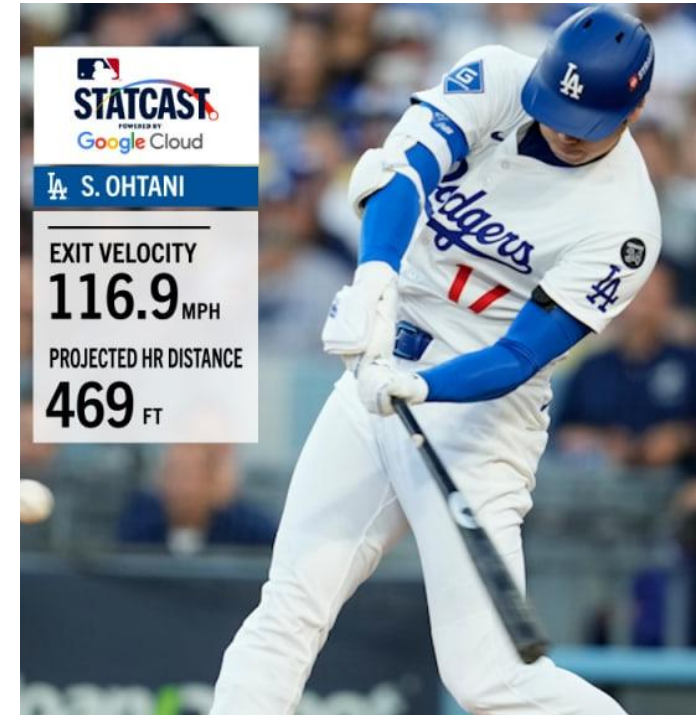
Some variables recording by Statcast

Pitching

- **Velocity:** Measures the peak velocity of a pitch
- **Spin rate:** The rate of spin by revolutions per minute of the ball

Hitting

- **Exit velocity:** Velocity of the ball off the bat
- **Launch angle:** The vertical angle at which the ball leaves the bat
- **Hit distance:** The distance on the ground of the actual landing point of any ball hit into play, ground/wall or contact with fielder



Some variables recording by Statcast

Base running

- **Lead distance:** the distance between the base and the runner's center of mass at the time the pitcher goes into his windup
- **Max speed:** the maximum speed at any point

Fielding

- **First step:** the time elapsed from time of bat-on-ball contact to the fielder's first movement toward the ball
- **First step efficiency:** The angle of deviation from a straight line to the ending point of a batted ball trajectory vs. the actual initial path taken toward the ball



Accessing the Statcast data

There are several ways to access the Statcast data in Python

1. Download from the baseball [Savant website](#)
2. Use the [pybaseball](#) python package
3. Load data into a database and access it using SQL

Let's try it in Jupyter!